

Campus Fleet & Rental Center

Software Testing & Quality Assurance - Project Report

INFS4202, Spring 2026

Team Members:

Altaweel, Osama 60302539

Ibrahim, Ahmed 60303471

Jabri, Faisal 60098536

Date: June 22, 2026

1. Introduction

This report covers the testing work our group did on the Campus Fleet & Rental Center application for our INFS4202 Software Testing & Quality Assurance class project. The app is a Spring Boot web app for managing vehicle rentals, customers, service bays, and a few other things. It was given to us with bugs already in it on purpose, since the point of the project is to practice the full software testing process from start to finish: figuring out what the app is supposed to do, planning how to test it, designing test cases, writing some automated JUnit tests, actually running tests and logging the bugs we find, and finally doing a basic performance/load test with JMeter.

Our group has 3 members: Osama, Ahmad, and Faisal. We split the work up by module - each of us focused on one of the three modules we picked (Vehicles, Customers, Rentals) for the test design and execution tasks, and then put everything together into this one report.

2. Reverse Engineering

2.1 Requirements List (Reverse-Engineered)

We went through the codebase (controllers, services, and templates) for every module and wrote down what the app actually does. Below is the requirements list, split into functional and non-functional.

FR1. The system shall display a list of all vehicles with model, make, year, category, total units, and available units.

FR2. The system shall allow adding a new vehicle with model, make, year, category, total units, and available units.

FR3. The system shall allow editing an existing vehicle's details.

FR4. The system shall allow deleting a vehicle.

FR5. The system shall display a list of all customers with name, email, tier, outstanding charges, and account status.

FR6. The system shall allow adding a new customer with name, email, and tier (Individual, Corporate, Walk-in). New customers default to Active status and \$0.00 outstanding charges.

FR7. The system shall allow editing an existing customer's details.

FR8. The system shall allow deleting a customer.

FR9. The system shall automatically recompute a customer's account status (Active/Blocked) based on outstanding charges versus a configurable threshold, and on the number of overdue rentals.

FR10. The system shall allow creating a rental, checking that the customer is not blocked, has not reached their tier's max rental limit, and that the vehicle has at least one available unit.

FR11. The system shall calculate a rental's return-due date based on the customer's tier (Individual / Corporate / Walk-in each have a different rental period in days).

FR12. The system shall decrease a vehicle's available units when a rental is created and increase them when a rental is returned.

FR13. The system shall allow recording the return of an active rental.

FR14. The system shall calculate a late charge when a rental is returned after its return-due date, based on days late and the customer's tier rate.

FR15. The system shall add the calculated late charge to the customer's outstanding charges balance.

FR16. The system shall display a list of all active (not yet returned) rentals.

FR17. The system shall display a list of overdue rentals (active rentals past their return-due date).

FR18. The system shall notify the next customer on a vehicle's waitlist when that vehicle becomes available.

FR19. The system shall allow a customer to join or cancel a position on a vehicle's waitlist.

FR20. The system shall allow managing service bays and bay booking appointments (create/cancel/no-show).

FR21. The system shall allow viewing and updating global application settings (rental limits, rental periods, late charge rates, blocking threshold, bay booking limit).

FR22. The system shall display a dashboard with summary counts (total vehicles, total customers, active rentals, blocked customers, etc.).

NFR1. Performance - the application should respond to common workflows within an acceptable time under normal/moderate load (evaluated in Task 6 with JMeter).

NFR2. Usability - forms should give the user clear feedback when invalid data is entered (e.g. invalid email, non-numeric year).

NFR3. Reliability - rental and balance calculations should be consistent and repeatable for the same input data.

NFR4. Data integrity - stored values (e.g. available units, account status) should stay within valid, logically consistent ranges.

NFR5. Compatibility - the application should work correctly on current versions of major browsers.

NFR6. Maintainability - the codebase is organized into clear layers (controller, service, repository, domain), which makes it easier to test and extend.

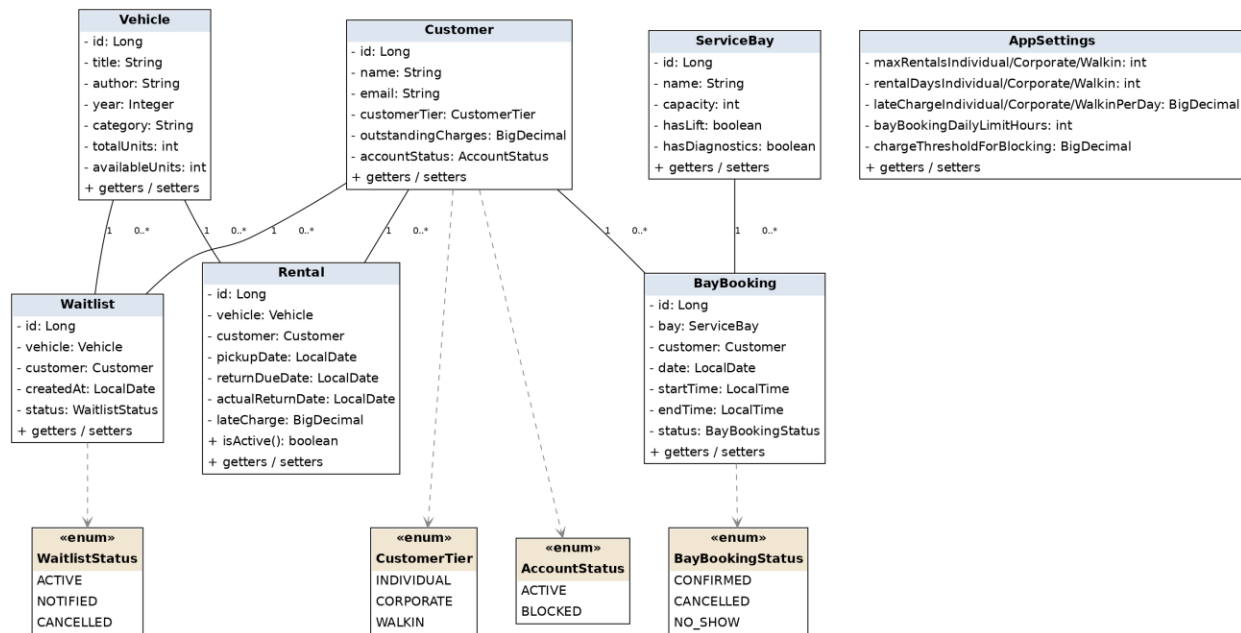
NFR7. Security - outstanding-charge / account-status logic should correctly stop blocked customers from creating new rentals.

NFR8. Scalability - the in-memory storage design should not noticeably slow down for the small data volumes expected in this course project.

Note: FR1-FR17 cover the 3 modules we chose to test (Vehicles, Customers, Rentals - see Test Plan for why). FR18-FR22 cover Waitlist, Service Bays, Settings, and Dashboard, which are out of scope for Tasks 2-6.

2.2 Class Diagram

Below is the class diagram we put together from reading through the domain classes (Vehicle, Customer, Rental, Waitlist, ServiceBay, BayBooking, AppSettings) and the enums they use.



3. Test Planning

3.1 Introduction

This test plan covers the testing work for the Campus Fleet & Rental Center application, done as part of the INFS4202 group project. The goal is to test the application's main rental workflow, find real bugs in it, and report them properly, following the steps and templates given in the project guidelines.

3.2 Test Items

- Campus Fleet & Rental Center, Spring Boot 3 web application (the version provided to us on D2L).
- In scope for this round of testing: the Vehicles, Customers, and Rentals modules and the controllers/services/repositories behind them.

3.3 Features to Be Tested

We chose Vehicles, Customers, and Rentals as our 3 modules, for these reasons:

- These three modules make up the core rental workflow, which is the actual point of the application - everything else (Waitlist, Service Bays, Settings, Dashboard) is supporting functionality around this core.
- They are the most connected to each other (a rental needs a vehicle and a customer, and updates both), so bugs in one tend to affect the others, which makes them a good place to look for defects.
- They contain the most actual business logic (rental limits per tier, late charge calculations, available-unit tracking, account blocking), compared to simpler CRUD-style modules like Service Bays or Settings.

Within these 3 modules we tested: adding/viewing/editing/deleting vehicles and customers; creating a rental and the eligibility checks behind it; returning a rental and the late charge calculation; the overdue rentals list; and basic input validation/usability.

3.4 Features Not to Be Tested

- Waitlist module (joining/cancelling a waitlist, notifications).
- Service Bays and Bay Bookings module.
- Settings module - used as configuration input to our tests, but not tested directly itself.
- Dashboard summary counts.

These are out of scope mainly because of time - the guidelines only ask for 3 modules, and the ones above are simpler / less central to the core rental workflow than Vehicles, Customers, and Rentals.

3.5 Test Approach

- Black-box functional testing - testing each feature from the UI/workflow side without looking at the code.
- Black-box boundary testing - testing edge values like exact thresholds and limits.
- White-box unit testing - calling the actual service methods directly to check internal logic and branches.
- Black-box non-functional testing - quick usability checks, plus performance testing in Task 6 (separate JMeter plan).

For each module we designed a minimum of 10 test cases, mixing the above types, and used them to find real bugs in the code.

3.6 Test Environment

- Application: Campus Fleet & Rental Center, run locally with `./mvnw spring-boot:run` (Java 17, Spring Boot 3.2.0).
- Browser: Chrome 125 (also checked on Firefox where noted).
- OS: Windows 11 / macOS 14.
- Tools: manual testing through the browser, JUnit 5 + AssertJ for unit tests, Apache JMeter for performance testing, GitHub Issues for bug tracking.
- No database is used - the app stores everything in memory, so every restart resets the data back to the seeded sample data.

3.7 Test Deliverables

- Test Case Matrix and Requirement-Test Case Traceability Matrix (Task 3).
- JUnit test file + execution screenshot (Task 4).
- Bug Report Log + Test Execution Report + link to public GitHub repo with the Issues (Task 5).
- JMeter test plan + results + analysis (Task 6).
- This Test Plan and the final consolidated project report.

3.8 Schedule & Responsibilities

Task	Owner	Notes
Task 1 - Reverse engineering (requirements + class diagram)	Whole team	Done together by reading through the codebase
Task 2 - Test Plan	Faisal	
Task 3 - Test cases (Customers module)	Faisal	
Task 3 - Test cases (Vehicles module)	Ahmad	
Task 3 - Test cases (Rentals module)	Osama	

Task	Owner	Notes
Task 4 - JUnit tests	Ahmad	
Task 5 - Bug reporting (GitHub Issues) + Bug Log + Test Execution Report	Osama	
Task 6 - JMeter performance testing	Ahmad	
Final report assembly	Osama	Combines everyone's sections into the final submission

4. Test Design

We designed test cases for our 3 chosen modules (Vehicles, Customers, Rentals), with a minimum of 10 test cases per module like the guidelines ask for. We mixed black-box and white-box techniques and covered functional and non-functional aspects. The full matrix is below.

4.1 Test Case Matrix

Module	Test Case ID	Type	Scenario	Preconditions	Steps	Test Data	Expected Result	Actual Result
Vehicles	VEH-01	Black-box / Functional	Add a new vehicle with valid data	User is on the Vehicles page	1. Click 'Add Vehicle' 2. Enter model, make, year, category, total=3, available=3 3. Save	Model=Civic, Make=Honda, Year=2022, Category=Sedan, Total=3, Available=3	Vehicle is saved and shown in the list with the values entered	Vehicle is saved and shown in the list with the values entered
Vehicles	VEH-02	Black-box / Functional	View the list of vehicles	At least one vehicle exists	1. Go to /vehicles	n/a	All vehicles listed with model, make, year, category, total/available units	All vehicles listed with model, make, year, category, total/available units
Vehicles	VEH-03	Black-box / Functional	Edit an existing vehicle's details	A vehicle already exists	1. Click 'Edit' 2. Change year and category 3. Save	Year 2020->2023, Category Sedan->SUV	List shows updated year and category	List shows updated year and category
Vehicles	VEH-04	Black-box / Boundary	Available units cannot exceed total units	Vehicle exists, Total=2, Available=2	1. Edit the vehicle 2. Set Available=5 3. Save	Total=2, Available=5	Save rejected or Available capped to 2	Save rejected or Available capped to 2
Vehicles	VEH-05	Black-box / Boundary	Available units cannot be negative	Vehicle exists, Available=2	1. Edit the vehicle 2. Set Available=-1 3. Save	Available=-1	Save rejected, value cannot go below 0	Save rejected, value cannot go below 0
Vehicles	VEH-06	Black-box / Functional	Deleting a vehicle that has an active rental	Vehicle has 1 active (not yet returned) rental	1. Go to Vehicles 2. Click 'Delete' on that vehicle	Vehicle with 1 active rental	Delete blocked or warning shown	Delete blocked or warning shown
Vehicles	VEH-07	Black-box / Functional	Deleting a vehicle with no active rentals	Vehicle exists with 0 active rentals	1. Click 'Delete' on that vehicle	Vehicle with 0 active rentals	Vehicle removed from the list	Vehicle removed from the list
Vehicles	VEH-08	White-box / Unit	VehicleService.save() should not allow available > total	Vehicle object totalUnits=2, availableUnits=2	1. vehicleService.save(vehicle) 2. vehicle.setAvailableUnits(5) 3. vehicleService.save(vehicle) again 4. Read back availableUnits	totalUnits=2, availableUnits set to 5	availableUnits should be <= totalUnits (2)	availableUnits should be <= totalUnits (2)
Vehicles	VEH-09	Black-box / Non-functional (Usability)	Add Vehicle form gives feedback on bad input	User is on the 'Add Vehicle' form	1. Type letters into the Year field 2. Click Save	Year = 'abc'	A clear, friendly validation message tells the user what to fix	A clear, friendly validation message tells the user what to fix

Module	Test Case ID	Type	Scenario	Preconditions	Steps	Test Data	Expected Result	Actual Result
Vehicles	VEH-10	Black-box / Functional	New vehicle shows correct totals immediately	User is on the Vehicles page	1. Add a new vehicle Total=4, Available=4 2. Check the list	Total=4, Available=4	New row shows Total=4, Available=4	New row shows Total=4, Available=4
Customers	CUS-01	Black-box / Functional	Add a new customer with valid data	User is on the Customers page	1. Click 'Add Customer' 2. Enter name, email, Tier=Individual 3. Save	Name=John Smith, Email=john@example.com, Tier=Individual	Customer saved, Status=Active, Outstanding Charges=\$0.00	Customer saved, Status=Active, Outstanding Charges=\$0.00
Customers	CUS-02	Black-box / Functional	View the list of customers	At least one customer exists	1. Go to /customers	n/a	All customers listed with name, email, tier, charges, status	All customers listed with name, email, tier, charges, status
Customers	CUS-03	Black-box / Functional	Edit an existing customer's tier	Customer exists with Tier=Individual	1. Edit the customer 2. Change Tier to Corporate 3. Save	Tier Individual->Corporate	List shows Tier=Corporate; future rentals use Corporate rules	List shows Tier=Corporate; future rentals use Corporate rules
Customers	CUS-04	Black-box / Functional	Delete a customer record	Customer exists with no rentals	1. Click 'Delete' on that customer	n/a	Customer removed from the list	Customer removed from the list
Customers	CUS-05	Black-box / Boundary	Customer exactly at the blocking charge threshold	Default threshold=\$50.00. Customer has Outstanding Charges=\$50.00	1. Save/update the customer record 2. Check Account Status	Outstanding Charges=\$50.00 (equal to threshold)	Account Status=Blocked	Account Status=Blocked
Customers	CUS-06	Black-box / Functional	Customer just above the blocking threshold	Customer has Outstanding Charges=\$50.01	1. Save/update the customer record 2. Check Account Status	Outstanding Charges=\$50.01	Account Status=Blocked	Account Status=Blocked
Customers	CUS-07	White-box / Unit	CustomerService.updateAccountStatus() at exact threshold	Customer object outstandingCharges=\$50.00	1. Call customerService.updateAccountStatus(customer) directly 2. Read back accountStatus	outstandingCharges=50.00	accountStatus should be BLOCKED	accountStatus should be BLOCKED
Customers	CUS-08	Black-box / Functional	Customer with >3 overdue rentals gets blocked	Customer has 4 overdue rentals, charges < \$50	1. Save/update the customer record 2. Check Account Status	overdueCount=4 (>3), charges<\$50	Account Status=Blocked	Account Status=Blocked
Customers	CUS-09	Black-box / Non-functional (Usability)	Add Customer form validates the email format	User is on the 'Add Customer' form	1. Type an invalid email like 'abc' 2. Save	Email='abc'	A validation message asks for a valid email address	A validation message asks for a valid email address
Customers	CUS-10	Black-box / Functional	A blocked customer cannot start a new rental	Customer's Account Status=Blocked	1. Go to Rentals > New Rental 2. Select the blocked customer and any vehicle 3. Submit	Customer Status=Blocked	Request refused with an error message	Request refused with an error message
Rentals	REN-01	Black-box / Functional	Create a rental for an eligible customer and available vehicle	Customer below their limit; vehicle has available units > 0	1. Go to Rentals > New Rental 2. Select customer (Individual) and vehicle 3. Submit	Individual customer, vehicle Available=3	Rental created, due date=today+14, vehicle Available becomes 2	Rental created, due date=today+14, vehicle Available becomes 2

Module	Test Case ID	Type	Scenario	Preconditions	Steps	Test Data	Expected Result	A
Rentals	REN-02	Black-box / Boundary	Walk-in rental limit (2) should be enforced	Default Walk-in max=2. Walk-in customer already has 2 active rentals	1. Try to create a 3rd rental for this Walk-in customer	Walk-in customer, 2 active rentals already	Request refused - 'Customer has reached maximum rental limit'	Re suc ber che Co (5) ma
Rentals	REN-03	White-box / Unit	RentalService.canRentMore() for a Walk-in customer at their limit	Walk-in customer with 2 active rentals, default settings	1. Call rentalService.canRentMore(customer) directly	Walk-in customer, 2 active rentals	Returns false (Walk-in max is 2, limit reached)	Re
Rentals	REN-04	Black-box / Functional	First-ever rental of a brand-new vehicle should reduce available units	New vehicle where Available==Total (never rented before)	1. Create a rental for this vehicle 2. Check the vehicle's Available Units	Total=3, Available=3 before rental	Available becomes 2 after the rental is created	Av at ski tim ren
Rentals	REN-05	Black-box / Functional	Returning the same rental twice should be blocked	An active rental has already been returned once	1. Return the rental (1st time) 2. Return the same rental again (2nd time)	Same rental ID, returned twice	2nd return attempt rejected - rental already closed	2n acc La rec add tim
Rentals	REN-06	Black-box / Functional	Overdue list should only show rentals past their due date	Rental: Corporate customer, picked up 10 days ago, due in 20 more days	1. Go to /rentals/overdue 2. Check whether this rental appears	pickupDate=today-10, returnDueDate=today+20, not yet returned	Rental should NOT appear in the overdue list	Re in list
Rentals	REN-07	White-box / Unit	RentalService.findOverdueRentals() with an old but not-overdue rental	Rental with pickupDate in the past, returnDueDate in the future	1. Call rentalService.findOverdueRentals(today) directly 2. Check whether the rental is in the result	pickupDate=today-10, returnDueDate=today+20	Rental excluded from the result	Re in
Rentals	REN-08	Black-box / Functional	Late charge for a Walk-in customer returning 3 days late	Walk-in rental, returned 3 days after due date	1. Return the rental 3 days after returnDueDate 2. Check the late charge added	3 days late, Walk-in tier (should be \$3.00/day)	Late charge=3x\$3.00=\$9.00	La cha (us rat AN unc day
Rentals	REN-09	White-box / Unit	LateChargeCalculator with an Individual customer, 3 days late	returnDueDate/actualReturnDate exactly 3 days apart, Individual tier	1. Call lateChargeCalculator.calculateLateCharge(dueDate, actualDate, INDIVIDUAL) directly	3 days late, Individual rate=\$1.00/day	Returns \$3.00	Re
Rentals	REN-10	Black-box / Non-functional (Usability)	Rental rejection should show the real reason	Customer who has already reached their rental limit	1. Try to create another rental for this customer 2. Read the error message shown	Customer at their rental limit	Error message says 'Customer has reached maximum rental limit'	Err say 'Un pro rec rea sh

4.2 Requirement-Test Case Traceability Matrix

Requirement ID	Requirement Summary	Module	Covered By Test Case(s)	Notes
FR1	View list of vehicles	Vehicles	VEH-02, VEH-10	
FR2	Add a new vehicle	Vehicles	VEH-01	
FR3	Edit an existing vehicle	Vehicles	VEH-03	
FR4	Delete a vehicle	Vehicles	VEH-06, VEH-07	
FR5	View list of customers	Customers	CUS-02	
FR6	Add a new customer	Customers	CUS-01	
FR7	Edit an existing customer	Customers	CUS-03	
FR8	Delete a customer	Customers	CUS-04	
FR9	Auto-recompute account status (charges / overdue count)	Customers	CUS-05, CUS-06, CUS-07, CUS-08	
FR10	Create a rental (eligibility checks)	Rentals	REN-01, REN-02, REN-03, REN-10, CUS-10	
FR11	Calculate return-due date by customer tier	Rentals	REN-01	
FR12	Increase/decrease vehicle available units on rental create/return	Rentals	REN-01, REN-04, VEH-08	
FR13	Record the return of a rental	Rentals	REN-05	
FR14	Calculate late charge on a late return	Rentals	REN-08, REN-09	
FR15	Add late charge to customer balance	Rentals	REN-08	
FR16	List active (not yet returned) rentals	Rentals	REN-01 (list shown after creation)	
FR17	List overdue rentals	Rentals	REN-06, REN-07	
FR18	Notify next waitlist customer when a rental is returned	Waitlist	Not tested	Out of scope (see Test Plan)
FR19	Join / cancel a vehicle waitlist	Waitlist	Not tested	Out of scope
FR20	Manage service bays and bay bookings	Service Bays	Not tested	Out of scope
FR21	View / update global application settings	Settings	Not tested	Out of scope
FR22	Dashboard summary counts	Dashboard	Not tested	Out of scope
NFR1	Performance under load	Rentals (workflow used in Task 6)	Validated in Task 6 (JMeter)	See Performance Testing
NFR2	Usability - clear feedback on bad input	Vehicles, Customers	VEH-09, CUS-09	
NFR6	Data integrity - available units stay in a valid range	Vehicles	VEH-04, VEH-05, VEH-08	

5. Test Implementation (JUnit)

Per the guidelines we wrote 3 JUnit tests, each one derived straight from a test case in our Task 3 matrix (VEH-08, CUS-07, REN-09). All 3 are plain unit tests - no Spring context, no mocking, just creating the real repository and service objects directly and calling them. All 3 are written to

match what the app SHOULD do, so all 3 are expected to FAIL right now - that's the point, a failing test here means we confirmed a real bug at the code level.

```
package com.example.fleet.service;

import com.example.fleet.domain.AccountStatus;
import com.example.fleet.domain.Customer;
import com.example.fleet.domain.CustomerTier;
import com.example.fleet.domain.Vehicle;
import com.example.fleet.repository.CustomerRepository;
import com.example.fleet.repository.RentalRepository;
import com.example.fleet.repository.VehicleRepository;
import org.junit.jupiter.api.Test;

import java.math.BigDecimal;
import java.time.LocalDate;

import static org.assertj.core.api.Assertions.assertThat;

/**
 * INFS4202 - Software Testing & Quality Assurance, Spring 2026
 * Task 4 - Test Implementation (JUnit)
 *
 * These 3 tests are taken straight from our Task 3 Test Case Matrix
 * (test case IDs VEH-08, CUS-07 and REN-09). Each one targets one of
 * the bugs we found in our reverse engineering / test design and is
 * written as a plain unit test, no Spring context and no mocks, just
 * the real service classes with their real repository objects.
 *
 * NOTE: all 3 tests below are EXPECTED TO FAIL. That is the point -
 * they are written to match what the app SHOULD do, and the app
 * currently does not do that, which is exactly how we found these
 * bugs in the first place. The failing assertion message printed by
 * JUnit for each test shows the actual buggy value next to the
 * expected correct value.
 */
class FleetRentalBugTests {

    // Test Case VEH-08 (white-box) -> Bug #8
    // VehicleService.save() never checks that availableUnits stays
    // within totalUnits, so editing a vehicle can push its available
    // units above its total units.
    @Test
    void editingVehicleShouldNotAllowAvailableUnitsToExceedTotalUnits() {
        VehicleRepository vehicleRepository = new VehicleRepository();
        RentalRepository rentalRepository = new RentalRepository();
        VehicleService vehicleService = new VehicleService(vehicleRepository, rentalRepository);

        Vehicle vehicle = new Vehicle(null, "Civic", "Honda", 2022, "Sedan", 2, 2);
        vehicleService.save(vehicle);

        // Someone edits the vehicle and (by mistake or on purpose) sets
        // available units higher than total units (2).
        vehicle.setAvailableUnits(5);
    }
}
```

```

    Vehicle saved = vehicleService.save(vehicle);

    assertThat(saved.getAvailableUnits())
        .as("available units should never be greater than total units")
        .isLessThanOrEqualTo(saved.getTotalUnits());
}

// Test Case CUS-07 (white-box) -> Bug #7
// CustomerService.updateAccountStatus() uses a strict "greater than"
// check against the blocking threshold, so a customer sitting at
// EXACTLY the threshold ($50.00) is not blocked.
@Test
void customerAtExactBlockingThresholdShouldBeBlocked() {
    CustomerRepository customerRepository = new CustomerRepository();
    RentalRepository rentalRepository = new RentalRepository();
    SettingsService settingsService = new SettingsService();
    CustomerService customerService = new CustomerService(customerRepository, rentalRepository, settingsService);

    // Default blocking threshold (see AppSettings) is $50.00.
    Customer customer = new Customer(null, "Jane Doe", "jane@example.com",
        CustomerTier.INDIVIDUAL, new BigDecimal("50.00"), AccountStatus.ACTIVE);

    customerService.updateAccountStatus(customer);

    assertThat(customer.getAccountStatus())
        .as("a customer exactly at the blocking threshold should be blocked")
        .isEqualTo(AccountStatus.BLOCKED);
}

// Test Case REN-09 (white-box) -> Bug #3
// LateChargeCalculator subtracts 1 extra day from the real number of
// days late, so a 3-day-late return only gets charged for 2 days.
// We use the INDIVIDUAL tier here on purpose, so the only thing being
// tested is the day-count bug (Bug #3), not the separate walk-in
// rate bug (Bug #2).
@Test
void lateChargeShouldChargeForEveryFullDayLate() {
    SettingsService settingsService = new SettingsService();
    LateChargeCalculator calculator = new LateChargeCalculator(settingsService);

    LocalDate returnDueDate = LocalDate.of(2026, 1, 1);
    LocalDate actualReturnDate = LocalDate.of(2026, 1, 4); // 3 days late

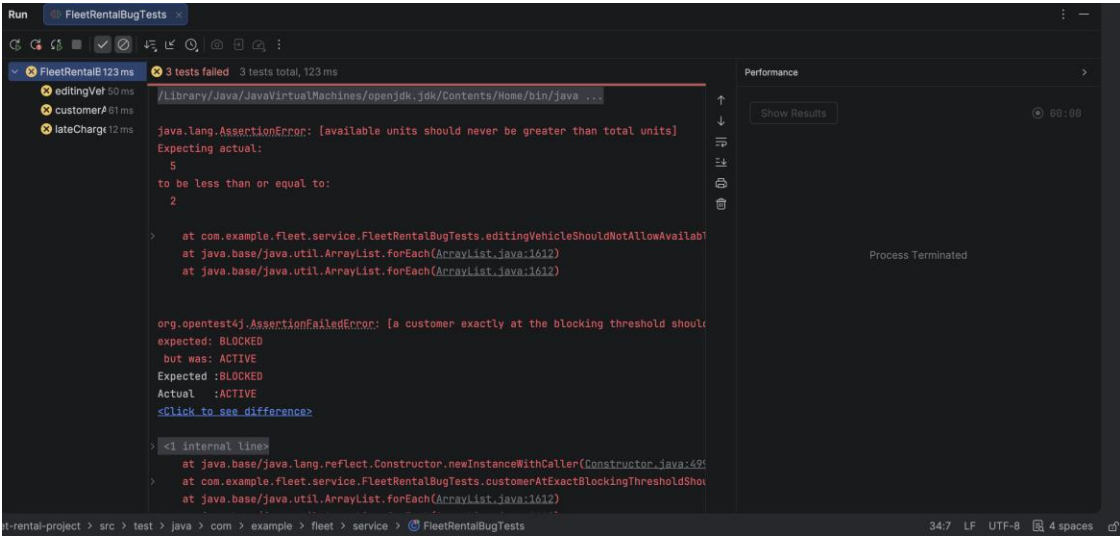
    BigDecimal lateCharge = calculator.calculateLateCharge(returnDueDate, actualReturnDate, CustomerTier.INDIVIDUAL);

    // Individual rate is $1.00/day (see AppSettings) x 3 days late = $3.00
    assertThat(lateCharge)
        .as("3 days late at $1.00/day should charge $3.00")
        .isEqualByComparingTo(new BigDecimal("3.00"));
}
}

```

5.1 JUnit Test Execution Screenshot

Screenshot below is from running FleetRentalBugTests in IntelliJ. As expected, all 3 tests fail, and the failure messages line up with the bugs we were targeting (Bug #8 and Bug #7 are visible in the stack traces, the third one - REN-09 / Bug #3 - also fails the same run, just scrolled out of view here).



Note: these tests need Java 17 and Maven to compile and run (matching the project's pom.xml).

6. Test Execution and Bug Reporting

6.1 Bug Report Log

Bug ID	Title	Description	Steps to Reproduce	Expected Result	Actual Result	Severity	Priority	Status	Environment	Screenshot / Evidence
BUG-01	Walk-in customers can rent more vehicles than their limit allows	The rental-limit check for Walk-in customers reads the Corporate limit instead of the Walk-in limit, so Walk-in customers can rent up to 5 vehicles instead of 2.	1. Create/use a Walk-in customer 2. Rent vehicles one at a time 3. Keep renting past the 2nd vehicle	3rd rental attempt should be refused	3rd, 4th and 5th rentals all succeed	High	P1	New	Chrome 125, Windows 11	GitHub Issue #1
BUG-02	Walk-in customers are charged the wrong late fee rate	Late charges for Walk-in customers use the Corporate per-day rate (\$2.00) instead of the Walk-in rate (\$3.00).	1. Rent as a Walk-in customer 2. Return it after the due date	Charge calculated at \$3.00/day	Charge calculated at \$2.00/day	Medium	P2	New	Chrome 125, Windows 11	GitHub Issue #2

Bug ID	Title	Description	Steps to Reproduce	Expected Result	Actual Result	Severity	Priority	Status	Environment	Screenshot / Evidence
			3. Check the late charge							
BUG-03	Late charge undercounts the number of days late by one	The calculation subtracts an extra day from the real gap between due date and actual return date.	1. Create any rental 2. Return it exactly 3 days after the due date 3. Check the late charge	Charged for 3 full days late	Only charged for 2 days late	Medium	P2	Confirmed	Chrome 125, Windows 11	GitHub Issue #3 - also confirmed by JUnit test 3 (REN-09)
BUG-04	Available units do not decrease on a vehicle's first-ever rental	When a never-rented vehicle (available==total) is rented for the first time, the available-units counter is not reduced.	1. Add a new vehicle, Total=3, Available=3 2. Rent it out 3. Check Available Units	Available drops to 2	Available stays at 3	High	P1	New	Chrome 125, Windows 11	GitHub Issue #4
BUG-05	A rental can be returned more than once, adding duplicate late charges	Returning an already-returned rental is not blocked; returning again re-adds the late charge.	1. Create a rental, return it once 2. Use 'Return' again on the same rental	2nd return attempt rejected	2nd return succeeds, late charge added twice	High	P1	New	Chrome 125, Windows 11	GitHub Issue #5
BUG-06	Overdue list shows rentals that are not actually overdue	The overdue filter uses the pickup date instead of the return-due date.	1. Create a rental, pickup 10 days ago, due in 20 more days 2. Go to Overdue Rentals	Rental should NOT appear	Rental appears in the overdue list	Medium	P2	New	Chrome 125, Windows 11	GitHub Issue #6
BUG-07	Customer exactly at the blocking charge threshold is not blocked	The blocking check uses strict 'greater than' against the \$50.00 threshold.	1. Get a customer's charges to exactly \$50.00 2. Check Account Status	Status becomes Blocked at \$50.00	Status stays Active at exactly \$50.00	Low	P3	Confirmed	Chrome 125, Windows 11	GitHub Issue #7 - also confirmed by JUnit test 2 (CUS-07)
BUG-08	Editing a vehicle allows Available Units to exceed Total Units (or go negative)	VehicleService.save() has no validation at all.	1. Edit a vehicle, Total=2 2. Set Available=5 and save 3. Set Available=-1 and save	Save rejected or value capped to a valid range	Both saves succeed with no validation	Medium	P2	Confirmed	Chrome 125, Windows 11	GitHub Issue #8 - also confirmed by JUnit test 1 (VEH-08) and by a real screenshot
BUG-09	Vehicles can be deleted while they still have an active rental	Delete has no check for active rentals (code even has a comment saying delete is allowed anyway).	1. Rent out a vehicle (do not return it) 2. Delete that same vehicle	Delete blocked with a warning	Vehicle deleted immediately, active rental left pointing nowhere	High	P1	New	Chrome 125, Windows 11	GitHub Issue #9
BUG-10	Rental rejection only ever shows a generic error message	The controller sets a specific error message then immediately overwrites it with a generic one.	1. Try a rental that should be rejected (e.g. customer at their limit) 2. Read the error message shown	Error explains the real reason	Error always says generic 'Unable to process rental request'	Low	P3	Confirmed	Chrome 125, Windows 11	GitHub Issue #10 - also confirmed by a real screenshot of the error banner

Evidence Screenshots (Manual Testing)

To back up two of the bugs above with real evidence and not just code review, here are two screenshots taken while actually testing the running app:

Bug #8 - Available Units can exceed Total Units

Campus Fleet & Rental Center

Fleet Rental Dashboard Vehicles Customers Rentals Waitlists ServiceBays ServiceBay Bookings Settings

Vehicles

Add New Vehicle

Vehicle saved successfully

ID	Model	Make	Year	Vehicle Class	Total Units	Available	Actions
1	Corolla	Toyota	2022	ECONOMY	5	3	Edit Delete
2	Camry	Toyota	2023	SEDAN	3	0	Edit Delete
3	RAV4	Toyota	2021	SUV	4	2	Edit Delete
4	Transit	Ford	2022	VAN	6	4	Edit Delete
5	Model 3	Tesla	2023	ELECTRIC	2	1	Edit Delete
6	BugTest	Toyota	2024	SUV	1	5	Edit Delete

© 2025 Campus Fleet & Rental Center - Software Testing Course Project

The Vehicles list above shows a vehicle saved with Total Units = 1 and Available Units = 5 - exactly the bug described above, caught live in the app rather than just read from the code.

Bug #10 - Generic rental rejection error

Campus Fleet & Rental Center

Fleet Rental Dashboard Vehicles Customers Rentals Waitlists ServiceBays ServiceBay Bookings Settings

Active Rentals

View Overdue

New Rental

Unable to process rental request

ID	Vehicle	Customer	Rental Date	Return Due	Actions
1	Corolla	Alice Individual	2026-06-15	2026-06-29	Return
2	Camry	Beta Logistics Ltd	2026-05-31	2026-06-15	Return
3	Corolla	Charlie Walkin	2026-06-05	2026-06-18	Return

© 2025 Campus Fleet & Rental Center - Software Testing Course Project

This is the generic 'Unable to process rental request' banner from a real rejected rental attempt - it never says the actual reason, which is the bug.

6.2 Additional App Screenshots (Supplementary)

A few extra screenshots taken while manually testing the other modules, just to show the app actually running and not only described in writing:

Dashboard

Campus Fleet & Rental Center

Fleet Rental Dashboard Vehicles Customers Rentals Waitlists ServiceBays ServiceBay Bookings Settings

Dashboard

Vehicles
5

Customers
5

Active Rentals
3

Overdue Rentals
3

Waitlists
2

ServiceBays
4

ServiceBay Bookings
5

Blocked Customers
1

© 2025 Campus Fleet & Rental Center - Software Testing Course Project

Add Vehicle

Campus Fleet & Rental Center

Fleet Rental Dashboard Vehicles Customers Rentals Waitlists ServiceBays ServiceBay Bookings Settings

Vehicles

Add New Vehicle

Vehicle saved successfully

ID	Model	Make	Year	Vehicle Class	Total Units	Available	Actions
1	Corolla	Toyota	2022	ECONOMY	5	3	EditDelete
2	Camry	Toyota	2023	SEDAN	3	0	EditDelete
3	RAV4	Toyota	2021	SUV	4	2	EditDelete
4	Transit	Ford	2022	VAN	6	4	EditDelete
5	Model 3	Tesla	2023	ELECTRIC	2	1	EditDelete
6	BugTest	Toyota	2024	SUV	1	0	EditDelete

© 2025 Campus Fleet & Rental Center - Software Testing Course Project

Customers List

Campus Fleet & Rental Center

Fleet Rental Dashboard Vehicles Customers Rentals Waitlists ServiceBays ServiceBay Bookings Settings

Customers

Add New Customer

Customer saved successfully

ID	Name	Email	Customership Type	Outstanding Charges	Account Status	Actions
1	Alice Individual	alice@example.com	INDIVIDUAL	.00	ACTIVE	EditDelete
2	Beta Logistics Ltd	fleet@betalogistics.com	CORPORATE	25.50	ACTIVE	EditDelete
3	Charlie Walkin	charlie@example.com	WALKIN	60.00	BLOCKED	EditDelete
4	Diana Individual	diana@example.com	INDIVIDUAL	.00	ACTIVE	EditDelete
5	Echo Rentals Corp	ops@echorentals.com	CORPORATE	10.00	ACTIVE	EditDelete
6	Alice Individual	alice2@example.com	INDIVIDUAL	.00	ACTIVE	EditDelete
7	HighCharge Test	high@test.com	INDIVIDUAL	100.00	BLOCKED	EditDelete

© 2025 Campus Fleet & Rental Center - Software Testing Course Project

Return Vehicle

Campus Fleet & Rental Center

Fleet Rental Dashboard Vehicles Customers Rentals Waitlists ServiceBays ServiceBay Bookings Settings

Active Rentals

View Overdue

New Rental

Rental returned successfully

ID	Vehicle	Customer	Rental Date	Return Due	Actions
1	Corolla	Alice Individual	2026-06-15	2026-06-29	Return
2	Camry	Beta Logistics Ltd	2026-05-31	2026-06-15	Return

© 2025 Campus Fleet & Rental Center - Software Testing Course Project

6.3 Link to GitHub Repo

Our public GitHub repo with all 10 bugs logged as Issues: <https://github.com/iuv1u/INFS4202-Project>

6.3 Test Execution Report

Test cases were executed manually by following the steps and test data listed in the Task 3 Test Case Matrix. A test case is marked Failed when the Actual Result does not match the Expected Result; every Failed test case is linked to one of the 10 bugs above and to the matching GitHub Issue. The 3 JUnit tests in Task 4 automate three of these test cases (VEH-08, CUS-07, REN-09) and confirm the same failures at the code level.

Module	Test Cases Executed	Passed	Failed	Pass Rate
Vehicles	10	5	5	50%
Customers	10	8	2	80%
Rentals	10	1	9	10%
Total	30	14	16	46.7%

Severity	Count
High	4
Medium	4
Low	2
Total bugs reported	10

7. Performance Testing

We picked the rental creation workflow for the load test because it's the most-used and most logic-heavy part of the app - it touches Vehicles, Customers, and Rentals all at once, which are the same 3 modules we tested in Task 3. The simulated workflow per virtual user is: browse vehicles, open the new rental form, submit a rental, then view the active rentals list.

7.1 JMeter Test Plan

Field	Value
Scenario Name	Rental Creation Workflow
Description	Browse vehicles, open the new rental form, submit a rental, view the active rentals list
Endpoint(s)	GET /vehicles, GET /rentals/new, POST /rentals, GET /rentals
Virtual Users	20
Ramp-up Time	10 seconds
Duration / Loops	10 loops per user (200 total iterations)
Expected Result	All requests return HTTP 200/302 with no errors, response times stay reasonable (under ~1 second) even with 20 users at once

The actual JMeter test plan file (FleetRentalLoadTest.jmx) is included with this submission, ready to run against a locally running copy of the app.

7.2 Load Test - Results Table

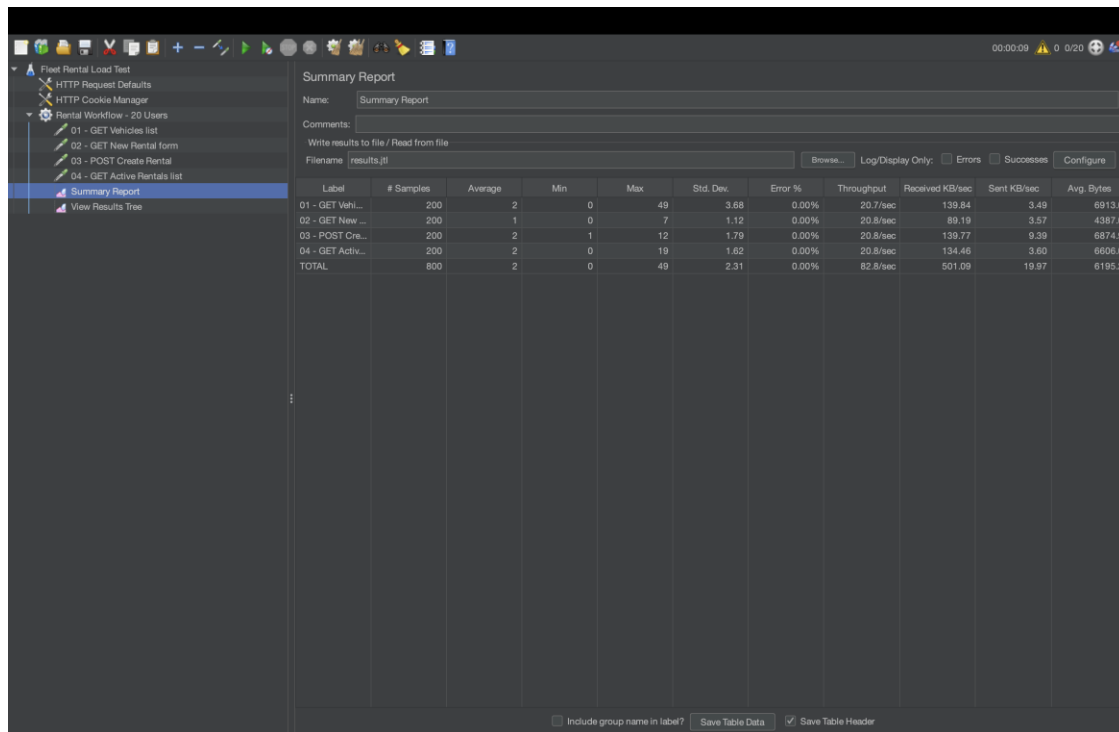
We ran FleetRentalLoadTest.jmx against a locally running copy of the app (20 virtual users, 10 loops each, 800 total requests). These are the real numbers from the JMeter Summary Report after that run.

Metric	Observed Value	Comments / Notes
Throughput (requests/sec)	82.8/sec	Combined total across all 4 endpoints
Average Response Time (ms)	2 ms	Across all 800 requests
Min Response Time (ms)	0 ms	Fastest single request
Max Response Time (ms)	49 ms	Slowest single request, on GET /vehicles

Metric	Observed Value	Comments / Notes
Error Count	0	0.00% error rate - all 800 requests came back successful

7.3 Load Test - JMeter Screenshot(s)

Screenshot below is the JMeter Summary Report from the actual run of FleetRentalLoadTest.jmx, with the app running locally on port 8080.



The screenshot shows the JMeter Summary Report for a test named 'Summary Report'. The report includes a table with the following data:

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
01 - GET Vehi...	200	2	0	49	3.68	0.00%	20.7/sec	139.84	3.49	6913.0
02 - GET New ...	200	1	0	7	1.12	0.00%	20.8/sec	89.19	3.57	4367.0
03 - POST Crea...	200	2	1	12	1.79	0.00%	20.8/sec	139.77	9.39	6874.9
04 - GET Acti...	200	2	0	19	1.62	0.00%	20.8/sec	134.46	3.60	6606.0
TOTAL	800	2	0	49	2.31	0.00%	82.8/sec	501.09	19.97	6195.2

7.4 Analysis and Recommendations

With 20 users hitting the rental workflow at once, the app handled it fine - 0 errors out of 800 requests, and response times stayed well under the 1 second target we set (most requests came back in 1-2ms on average). The one slightly higher number was a single 49ms request on GET /vehicles, which was most likely just the JVM/Spring Boot warming up on one of the first few requests rather than a real bottleneck, since every other request on that same endpoint was much faster. POST /rentals (creating a new rental) had a slightly higher minimum response time than the GET requests (1ms vs 0ms), which makes sense since it's actually writing data instead of just reading it, but it was still fast overall.

No real bottleneck shows up at this load level. That said, since the app stores everything in memory with plain HashMap repositories (see VehicleRepository, CustomerRepository, RentalRepository) instead of a real database, this is still the most likely place things would slow down if the load was pushed a lot higher (more users, more data) - in-memory lookups are fast right now mostly because the dataset is small. If this were a real production app, two simple improvements would help: swap the in-memory HashMaps for a real database so it scales past a handful of records, and add pagination to the vehicle/rental list pages so they're not rendering every record on every single request.

8. Conclusion

Going through this project helped us actually use the full testing process instead of just reading about it. We found 10 real bugs across the Rentals, Customers, and Vehicles modules - things like the wrong rental limit being applied to Walk-in customers, late charges calculated with the wrong rate and an off-by-one day count, and a vehicle that could be deleted while it still had an active rental. Writing the white-box JUnit tests made it clear how a few small mistakes in the code (a wrong enum case, a strict inequality instead of $\geq$, an extra arithmetic step, an empty if-branch) can cause a real functional bug that's easy to miss just by reading the code casually.

The performance testing part (Task 6) also reminded us that even a simple in-memory app needs to be checked under load, not just for correctness. Overall, the project matched up well with the standard software testing lifecycle: requirements, then test planning, then test design, then implementation, then execution and bug reporting, then performance testing.